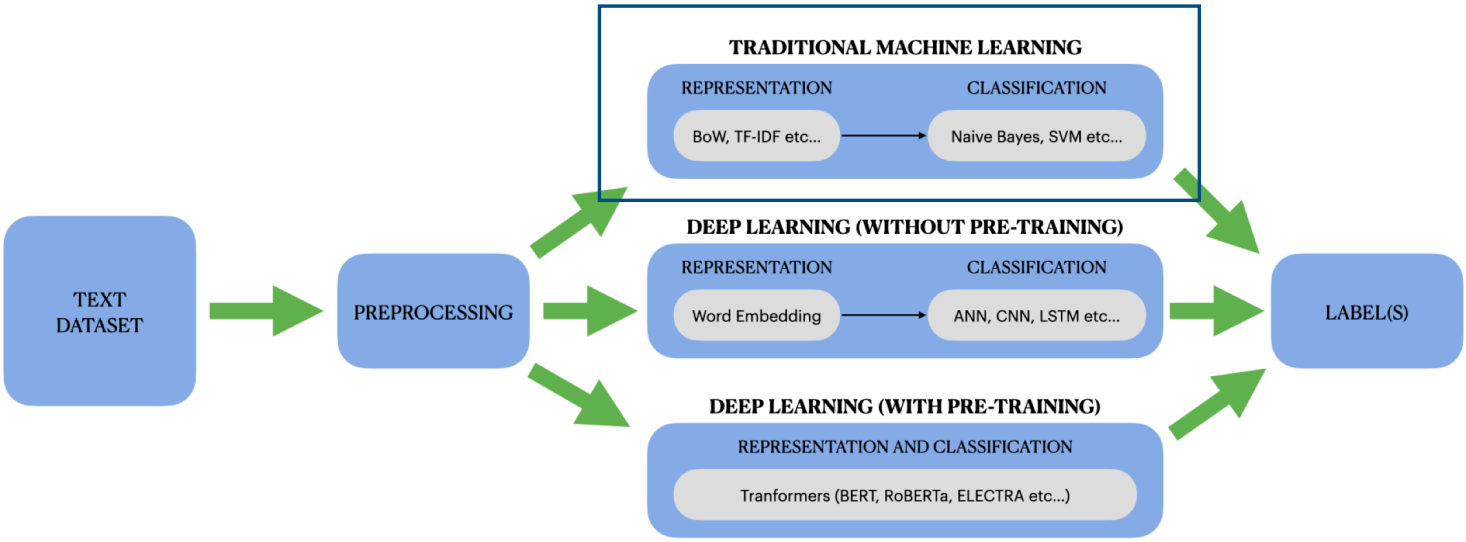
# Text Classification

CSEC 6 - Natural Language Processing




https://doi.org/10.48550/arXiv.2501.00174

# Text Representation

# Text representation models

convert t**ext data** into a **numerical vector space**, significantly influencing the performance of subsequent learning tasks.

# One-hot encoding

each categorical value is converted into a new categorical column, and a binary value of 1 or 0 is assigned to these columns.

*"Like a rolling stone"*

DICTIONARY

| KEY | WORD |
|---|---|
| 1 | Like |
| 2 | a |
| 3 | rolling |
| 4 | stone |

ONE-HOT ENCODING

| | | | | |
|---|---|---|---|---|
| Like | 1 | 0 | 0 | 0 |
| a | 0 | 1 | 0 | 0 |
| rolling | 0 | 0 | 1 | 0 |
| stone | 0 | 0 | 0 | 1 |

# Bag of Words (BoW)

BoW creates a vector representation of a document by counting the frequency of terms within the text, a technique also known as a vector space model.

*"As Long As You Love Me"*

DICTIONARY

| KEY | WORD |
|---|---|
| 1 | As |
| 2 | Long |
| 3 | You |
| 4 | Love |
| 5 | Me |

BoW ENCODING

| | | | | |
|---|---|---|---|---|
| 2 | 1 | 1 | 1 | 1 |

# Term Frequency-
# Inverse Document Frequency
# (TF-IDF)

a statistical method used in natural language processing and information retrieval to evaluate how important a word is to a document in relation to a larger collection of documents.

# Term Frequency (TF)

measures how often a word appears in a document. A higher frequency suggests greater importance. If a term appears frequently in a document, it is likely relevant to the document's content.

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

# Inverse Document Frequency (IDF)

Reduces the weight of common words across multiple documents while increasing the weight of rare words. If a term appears in fewer documents, it is more likely to be meaningful and specific.

$$IDF(t, D) = \log \frac{\text{Total number of documents in corpus } D}{\text{Number of documents containing term } t}$$

# Term Frequency-Inverse Document Frequency (TF-IDF)

$$\text{TF - IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

$\texttt{tf}(t, d)$

| | blue | bright | can | see | shining | sky | sun | today |
|---|---|---|---|---|---|---|---|---|
| 1 | 1/2 | 0 | 0 | 0 | 0 | 1/2 | 0 | 0 |
| 2 | 0 | 1/3 | 0 | 0 | 0 | 0 | 1/3 | 1/3 |
| 3 | 0 | 1/3 | 0 | 0 | 0 | 1/3 | 1/3 | 0 |
| 4 | 0 | 1/6 | 1/6 | 1/6 | 1/6 | 0 | 1/3 | 0 |

x

$\texttt{idf}(t, D)$

| | blue | bright | can | see | shining | sky | sun | today |
|---|---|---|---|---|---|---|---|---|
| | 0.602 | 0.125 | 0.602 | 0.602 | 0.602 | 0.301 | 0.125 | 0.602 |

- TF-IDF: Multiply TF and IDF scores, use to rank importance of words within documents
  - Most important word for each document is highlighted

$\texttt{tfidf}(t, d, D) = \texttt{tf}(t, d) \cdot \texttt{idf}(t, D)$

| | blue | bright | can | see | shining | sky | sun | today |
|---|---|---|---|---|---|---|---|---|
| 1 | **0.301** | 0 | 0 | 0 | 0 | 0.151 | 0 | 0 |
| 2 | 0 | 0.0417 | 0 | 0 | 0 | 0 | 0.0417 | **0.201** |
| 3 | 0 | 0.0417 | 0 | 0 | 0 | **0.100** | 0.0417 | 0 |
| 4 | 0 | 0.0209 | **0.100** | **0.100** | **0.100** | 0 | 0.0417 | 0 |

# Quiz 3

Calculate the TF-IDF score for all the words in the collection of three short book descriptions below

- Doc 1: "The brave knight fought the dragon."
- Doc 2: "The dragon lives in a cave."
- Doc 3: "The knight rescues the princess."

$$TF(t,d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

$$IDF(t,D) = \log \frac{\text{Total number of documents in corpus } D}{\text{Number of documents containing term } t}$$

$\texttt{tf}(t,d)$

| | blue | bright | can | see | shining | sky | sun | today |
|---|---|---|---|---|---|---|---|---|
| 1 | 1/2 | 0 | 0 | 0 | 0 | 1/2 | 0 | 0 |
| 2 | 0 | 1/3 | 0 | 0 | 0 | 0 | 1/3 | 1/3 |
| 3 | 0 | 1/3 | 0 | 0 | 0 | 1/3 | 1/3 | 0 |
| 4 | 0 | 1/6 | 1/6 | 1/6 | 1/6 | 0 | 1/3 | 0 |

x

$\texttt{idf}(t,D)$

| blue | bright | can | see | shining | sky | sun | today |
|---|---|---|---|---|---|---|---|
| 0.602 | 0.125 | 0.602 | 0.602 | 0.602 | 0.301 | 0.125 | 0.602 |

- TF-IDF: Multiply TF and IDF scores, use to rank importance of words within documents
  - Most important word for each document is highlighted

$\texttt{tfidf}(t,d,D) = \texttt{tf}(t,d) \cdot \texttt{idf}(t,D)$

| | blue | bright | can | see | shining | sky | sun | today |
|---|---|---|---|---|---|---|---|---|
| 1 | **0.301** | 0 | 0 | 0 | 0 | 0.151 | 0 | 0 |
| 2 | 0 | 0.0417 | 0 | 0 | 0 | 0 | 0.0417 | **0.201** |
| 3 | 0 | 0.0417 | 0 | 0 | 0 | **0.100** | 0.0417 | 0 |
| 4 | 0 | 0.0209 | **0.100** | **0.100** | **0.100** | 0 | 0.0417 | 0 |

# Quiz 3

| TF | a | brave | cave | dragon | fought | in | knight | lives | princess | rescues | the |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Doc 1** | 0.000 | 0.167 | 0.000 | 0.167 | 0.167 | 0.000 | 0.167 | 0.000 | 0.000 | 0.000 | 0.333 |
| **Doc 2** | 0.167 | 0.000 | 0.167 | 0.167 | 0.000 | 0.167 | 0.000 | 0.167 | 0.000 | 0.000 | 0.167 |
| **Doc 3** | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.200 | 0.000 | 0.200 | 0.200 | 0.400 |

| | a | brave | cave | dragon | fought | in | knight | lives | princess | rescues | the |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **IDF Score** | 0.4771 | 0.4771 | 0.4771 | 0.1761 | 0.4771 | 0.4771 | 0.1761 | 0.4771 | 0.4771 | 0.4771 | 0.0000 |

# Quiz 3

| TF-IDF | a | brave | cave | dragon | fought | in | knight | lives | princess | rescues | the |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Doc 1** | 0.00000 | 0.07952 | 0.00000 | 0.02935 | 0.07952 | 0.00000 | 0.02935 | 0.00000 | 0.00000 | 0.00000 | 0.00000 |
| **Doc 2** | 0.07952 | 0.00000 | 0.07952 | 0.02935 | 0.00000 | 0.07952 | 0.00000 | 0.07952 | 0.00000 | 0.00000 | 0.00000 |
| **Doc 3** | 0.00000 | 0.00000 | 0.00000 | 0.00000 | 0.00000 | 0.00000 | 0.03522 | 0.00000 | 0.09542 | 0.09542 | 0.00000 |

# Text Classification

# Text classification

task of categorizing text documents into predefined classes.

# Example: Sentiment analysis

*I love it so much! The mic works great!!!! I use it for online live classes, cosplay, and to look cute!! The lightup feature really works great! The app also works great too! The sound sounds amazing too! I just wish it had a case for when I travel.*

positive

*Not durable. The cord came apart from the audio adjuster. The saddest part is that happens only two months after it was purchased, and no force was applied. Definitely, I will not purchase and I do not recommend the item.*

negative

- Can also add a "***neutral***" class, or **different gradations** – e.g. strongly positive, positive, neutral, negative, strongly negative
- Can also predict **emotion labels**: "angry", "happy", "sad", ...

# Example: Spam detection

*Hello, we are pleased to confirm your order 412116 has been received and will be processed shortly. We will email you an update when we have shipped it. Please see below for the order details and the total price.*

not spam

*Greetings, we have checked your website and it looks like your website has serious SEO issues. We can get you into the TOP3 search results for a very affordable price. Please contact us on Whatsapp*

spam

# Example: Fake news detection

*The Goldbergs Actor Jackson Odell Died of an Accidental Drug Overdose*

*The cause of actor Jackson Odell's death has been revealed: The Goldbergs star had suffered an accidental drug overdose. The 20-year-old, who played Ari Caldwell on the ABC comedy series [...]*

real news

*Pregnant Kourtney Kardashian Hangs out With Justin Bieber Amid Baby Daddy Drama*

*You would think the last thing Kourtney Kardashian needs right now is more guy drama — but that didn't stop the pregnant reality star from stepping out with her former flame Justin Bieber! [...]*

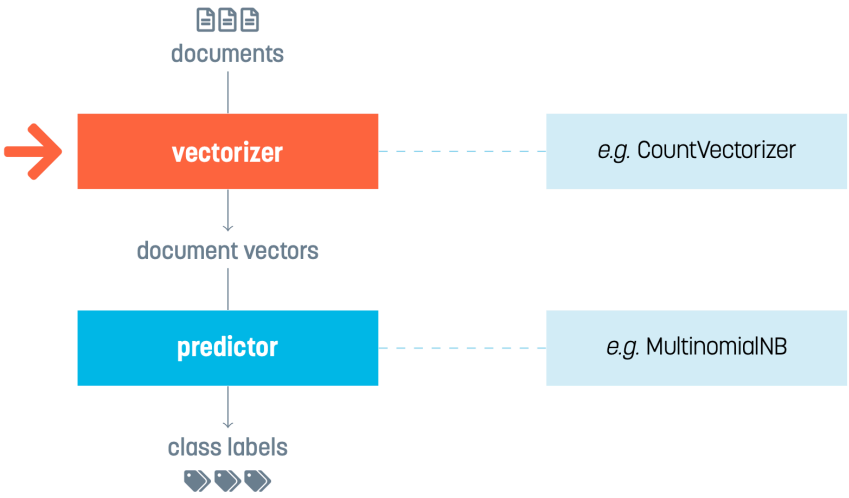
fake news

# Example: Topic classification

*It took them an hour of huffing and puffing, but Arsenal did something at Stamford Bridge they hadn't managed since September – they scored an away goal in the Premier League.*

- ✗ Business
- ✗ Politics
- ✗ Technology
- ✓ **Sports**
- ✗ Entertainment

# Text classification pipeline

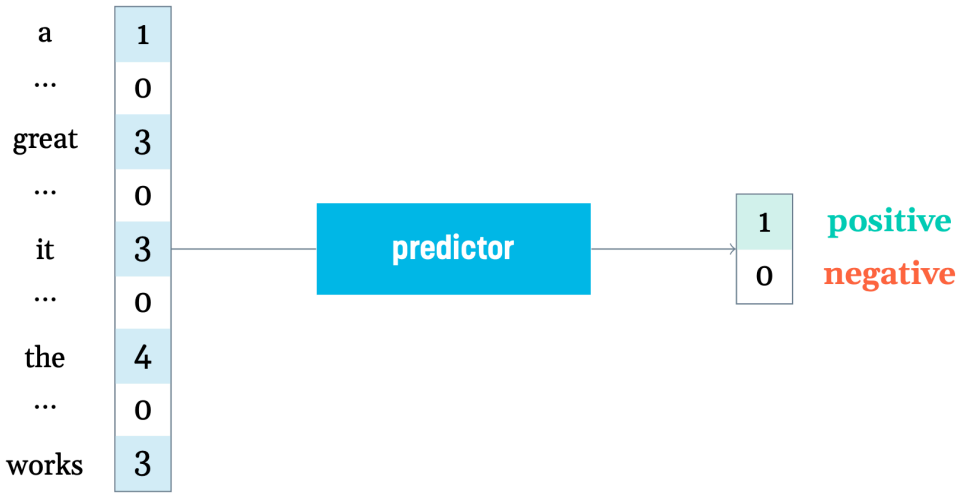


# Documents as count vectors: Bag of words

*It is a truth universally acknowledged, that a single man in possession of a good fortune must be in want of a wife.*

```
{'acknowledged': 1, 'be': 1, 'fortune': 1, 'good': 1, 'in': 2, 'is': 1,
 'it': 1, 'man': 1, 'must': 1, 'of': 2, 'possession': 1, 'single': 1,
 'that': 1, 'truth': 1, 'universally': 1, 'want': 1, 'wife': 1}
```

# Sentiment analysis with BoW: pipeline



# Drawbacks of count-based document representations

1. Count-based document representations can easily yield tens of thousands of features.

   - computational challenge

   - data sparsity — *most of the entries are zero*

2. Count-based representations are very limited in capturing dependencies between words in a sentence.

   - “I did not have a good experience with this product”

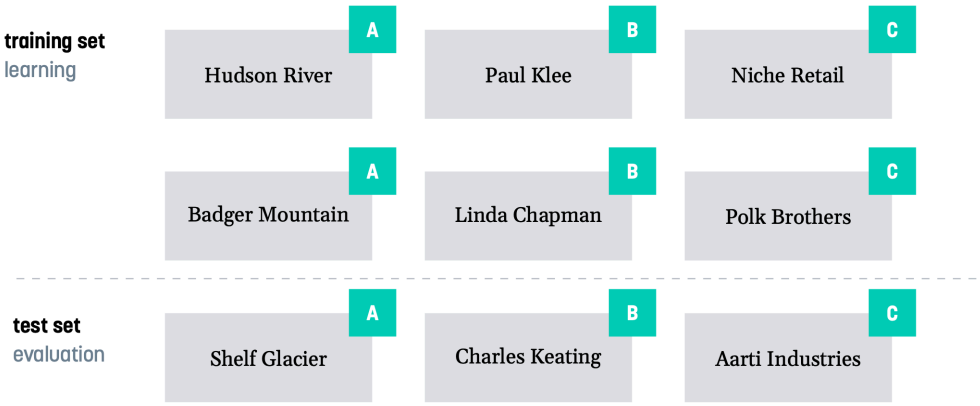
# n-gram models

One **solution** to problem 2 is to represent $n$**-grams** instead.

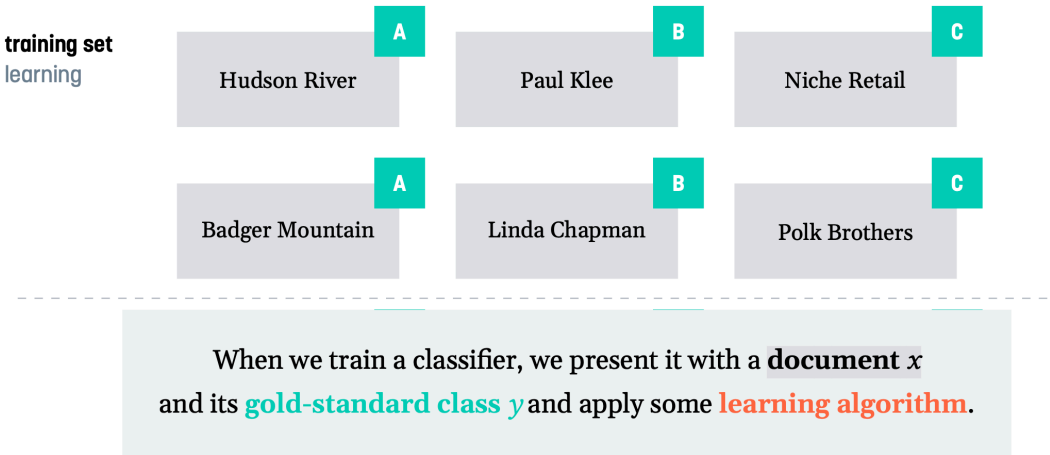
| | | *"I do not recommend the item"* |
|---|---|---|
| Unigrams | $n = 1$ | *I, do, not, recommend, the, item* |
| Bigrams | $n = 2$ | *I do, do not, not recommend, recommend the, the item* |
| Trigrams | $n = 3$ | *I do not, do not recommend, not recommend the, recommend the item* |

- **But**: Very low frequency counts the higher $n$ gets. (Problem 1 gets worse...)

# Classification as machine learning



# Training and testing



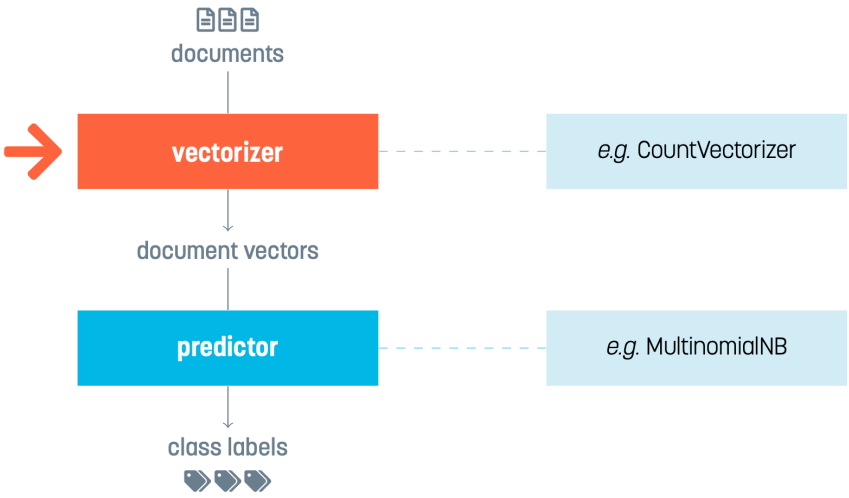

When we train a classifier, we present it with a **document** $x$ and its **gold-standard class** $y$ and apply some **learning algorithm**.

# Training and testing

When we evaluate a classifier, we present it with a **document** $x$ and compare the **predicted class** with the **gold-standard class** $y$.

**test set**
evaluation

# Statistical text classification pipeline



# Choosing a machine learning classifier

- **No** single type of classifier ***works best*** across **all possible scenarios**.

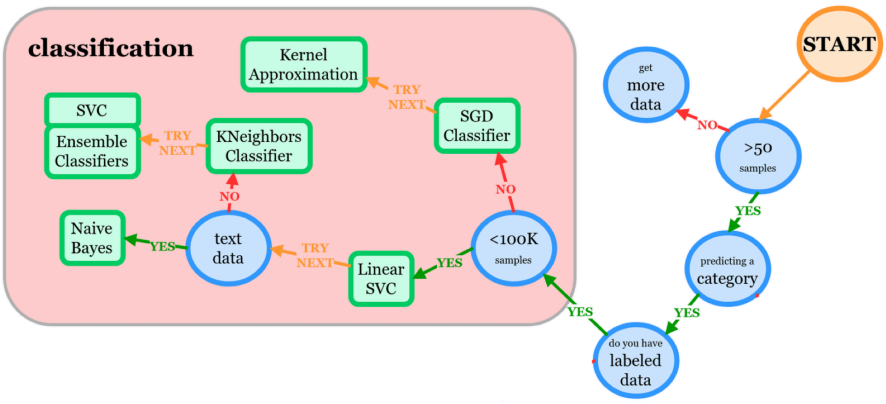
- Hand-point-right Libraries like ***scikit-learn*** make it very easy to "switch out" one classifier for another:

```
clf = make_pipeline(
          CountVectorizer(),
          MultinomialNB()
      )
clf.fit(X, y)
...
```

DecisionTreeClassifier
LogisticRegression
RidgeClassifier
SGDClassifier
SVC
...

# Choosing a machine learning classifier

You can choose any classification model (rather than e.g. regression algorithms).



# Naive Bayes

- **Naive Bayes** is a simple probabilistic classifier.
  - Models $P$(class, document), then infers $P$(class|document) via Bayes' rule.
- It is "*naive*" in the sense that it makes strong (= unrealistic) independence assumptions about probabilities.
  - Its estimated probability values are usually not very accurate.
- For text classification, you typically want to use **MultinomialNB** from scikit-learn.

# How to evaluate text classifiers?

- **Accuracy**: the proportion of documents for which the classifier was “correct”
  - Can be problematic/misleading when the classes are highly imbalanced
- **Precision, Recall, F1-score**: metrics that “zoom in” on the classifier’s performance with regard to a specific class
  - Take class imbalance into account
  - To report a single score: macro- vs. micro-averaging
- **Confusion matrix**: commonly-used tool to visualize classification performance

# How to find the best classifier?

- It is common to **try out different classifiers** in the process of searching for the "best" classifier for your task.

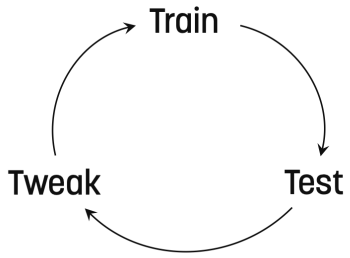
- You can also **tune the hyperparameters** of one classifier to find the best ones.

  - "Hyperparameter": A parameter of a model that you need to set before training

  - Example: Naive Bayes has a "smoothing constant $\alpha$"

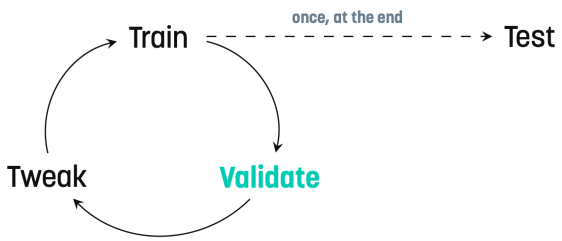
# A problem when searching for the best classifier

- If you train & test repeatedly, your test set effectively becomes part of the training procedure!



# Validating

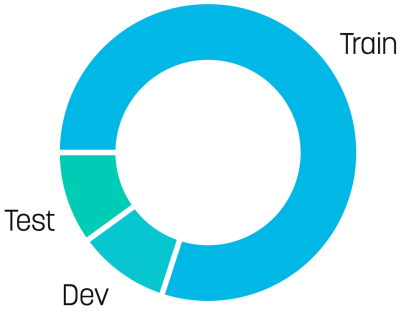
- Use a separate **validation set**, also called **development set**, while tweaking the model.
  - An alternative to this is using **cross-validation**.




- Use the test set only for the **final evaluation**.

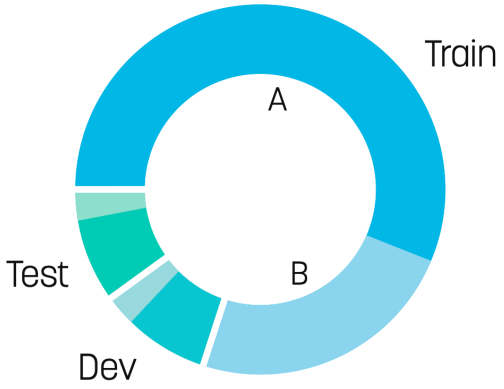
# Creating a data split

- **Randomly shuffle & split** your data into train/dev/test partitions.
  - Typically, you want to ***reserve most of your data for training***.



# Creating a data split

- **Stratified sampling** is a method of shuffling & splitting so that classes are represented equally in each split.



# Thank you!

*Text Classification*

**CSEC 6 - Natural Language Processing**

**Joseph Jessie S. Oñate, MSc.**

*Assistant Professor IV*